A PRACTICAL FIELD GUIDE

10 System Design Case Studies

Real problems, worked end-to-end — from requirements and back-of-the-envelope estimates to architecture and the hard trade-offs that actually decide the design.

Requirements → Scale → Design

Deep dives on the hard parts

Trade-offs & lessons

Designed as the sequel to your 10-day system design basics — now think like an engineer, not a memori

READ FIRST

How to use this book

The goal is judgment, not answers to memorize.

You already know the vocabulary: load balancers, caches, sharding, queues. This book is the next step — watching those pieces get **chosen and combined under pressure**. That choosing is the actual skill of a system engineer.

Every case study follows the same six-step spine. Learn the spine, and you can attack any problem you have never seen before — which is exactly what a real interview or a real project throws at you.

Step	The question it answers
1. Requirements	What must it do (functional) and how well (non-functional: scale, latency, availability, consistency)?
2. Estimate the scale	How many requests/sec, how much storage, how much bandwidth? Numbers decide the architecture.
3. API & data model	What are the core operations and how is data shaped and accessed?
4. High-level design	Draw the boxes and arrows. Get a working end-to-end path first.
5. Deep dive	Attack the 1–2 genuinely hard sub-problems. This is where the design is won or lost.
6. Trade-offs & lessons	Name what you gave up for what you gained. There is no free lunch, only a chosen one.

HOW TO READ FOR MAXIMUM THINKING

Before reading each solution, cover it and spend five minutes designing it yourself. Then compare. The *gap* between your design and the worked one — and the reasons behind the differences — is where the real learning lives. Passive reading builds recognition; active designing builds judgment.

Watch for three recurring boxes throughout: **Key insight** (the idea that unlocks the problem), **Trade-off** (the fork in the road and why we took one branch), and **Pitfall** (the mistake that looks fine until it fails in production).

The ten cases

Ordered roughly by conceptual build-up — each introduces tools the later ones reuse.

01	**URL Shortener** — TinyURL / Bitly	*hashing, base62, read-heavy caching*
02	**Rate Limiter** — API abuse protection	*token bucket, sliding window, atomic counters*
03	**Unique ID Generator** — IDs across shards	*Snowflake, clock skew, coordination*
04	**News Feed / Timeline** — Twitter	*fan-out on write vs read, the celebrity problem*
05	**Chat System** — WhatsApp / Messenger	*WebSockets, presence, delivery guarantees*
06	**Notification System** — push / SMS / email	*queues, retries, at-least-once, dedup*
07	**Web Crawler** — search-engine ingest	*URL frontier, politeness, dedup, traps*
08	**Distributed Key-Value Store** — Dynamo	*consistent hashing, quorum, CAP, vector clocks*
09	**Video Streaming** — YouTube / Netflix	*transcoding pipeline, CDN, adaptive bitrate*
10	**Ride-Sharing / Nearby** — Uber	*geospatial indexing, matching, location writes*

★ **Appendix** — The universal framework & toolbox — *a checklist for any problem*

TOOLBOX

The building blocks, in one page

The recurring pieces every case study reaches for.

You will see these terms constantly below. Here is the one-line mental model for each so the case studies read fast.

Block	What it buys you — and its cost
Load balancer	Spreads traffic across many servers; lets you scale *horizontally* and survive a dead node. Needs health checks.
Cache (Redis / Memcached)	Keeps hot data in memory for microsecond reads. Cost: it can go *stale*; you must pick an eviction (LRU) and invalidation policy.
CDN	Copies static/large content to edge servers near users. Slashes latency and origin load. Cost: money and cache-freshness.
Sharding (partitioning)	Splits one giant dataset across many DBs by a *shard key*. Enables scale beyond one machine. Cost: cross-shard queries and hot shards.
Replication	Copies of data on several nodes for availability + read scaling. Cost: keeping replicas *consistent*.
Consistent hashing	Maps keys to nodes on a ring so adding/removing a node reshuffles only ~1/N of keys, not everything.
Message queue (Kafka / SQS)	Decouples producers from consumers; absorbs bursts; enables retries and async work. Cost: eventual, not instant, processing.
Bloom filter	Tiny probabilistic set: “definitely not seen” or “maybe seen.” Huge memory savings when false positives are tolerable.
SQL vs NoSQL	SQL: relations, joins, transactions, strong consistency. NoSQL: simple access patterns at massive scale, flexible schema, tunable consistency.

CAP THEOREM — THE LAW BEHIND HALF THESE DECISIONS

When the network partitions (and at scale it *will*), a distributed store can keep either **C**onsistency (every read sees the latest write) or **A**vailability (every request gets an answer) — not both. Choosing **AP** (stay up, allow slightly stale reads) vs **CP** (refuse rather than serve stale) is the single most repeated fork in this book.

CASE 01

Design a URL Shortener

TinyURL, Bitly — the classic warm-up that teaches read-heavy scaling.

The problem

Turn a long URL like `https://example.com/articles/2026/very-long-path?ref=x` into a short one like `short.ly/aQ9bXk2`. When anyone visits the short link, redirect them to the original. Sounds trivial — but at scale it forces real decisions about ID generation, storage choice, and caching.

Step 1 — Requirements

Functional

- Shorten a long URL to a unique short key.
- Redirect a short key to the original URL.
- Optional: custom alias, expiration, click analytics.

Non-functional

- **Read-heavy** — redirects vastly outnumber creations (~100:1).
- Very low latency on redirect (it's in the user's critical path).
- High availability; keys must not be guessable/enumerable.

Step 2 — Estimate the scale

Writes: say 100M new URLs/day → 100M / 86,400s ≈ **~1,160 writes/sec**.

Reads: at 100:1 → **~116,000 reads/sec**. This ratio is the headline: *optimize the read path above all.*

Storage: keep records 5 years → 100M × 365 × 5 ≈ 182B rows. At ~500 bytes each → **~91 TB**. Too big for one machine → we will shard.

Step 3 — How long is the short key?

Use **base62** characters: `a–z A–Z 0–9` = 62 symbols, all URL-safe. How many characters do we need to cover 182B URLs?

```
62^6 =  56.8 billion   (too few for 182B)
62^7 =   3.52 trillion  (comfortably enough)
  =>  a 7-character key is the sweet spot.
```

Step 4 — Generating the key (the real decision)

Option A: Hash the long URL

Take `MD5(longURL)`, base62-encode it, keep the first 7 chars. Simple, but two different URLs can collide on the same 7 chars. You must detect the collision and retry (e.g. append a salt

and rehash). Also, the same URL always maps to the same key — sometimes desirable, sometimes not.

Option B: Counter → base62 (preferred)

Keep a global counter. Each new URL takes the next integer, and you base62-encode that integer to get the key. **No collisions ever**, because every integer is unique. The catch: a raw counter makes keys sequential and enumerable (`...aQ9bXk1`, `...aQ9bXk2`), which leaks how many links exist and lets people scrape. Fix by scrambling the integer or drawing from a pre-generated pool.

KEY INSIGHT — THE KEY GENERATION SERVICE (KGS)

Don't generate keys at request time. Run a small **KGS** that pre-computes billions of unique base62 keys offline and stores them in a "used" and "unused" set. On a write, a server grabs a *batch* of unused keys into memory and hands them out one by one. This removes collision checks from the hot path entirely and makes creation blazing fast.

PITFALL — THE KGS AS A SINGLE POINT OF FAILURE

If the one KGS dies mid-batch, keys held in its memory are lost (acceptable — the key space is huge) but the service stops. Run a **standby replica** and have each app server pre-fetch a batch so a brief KGS outage doesn't stall writes.

Step 5 — Data model & storage

The access pattern is dead simple: `get(shortKey) → longURL`. There are no joins, no complex queries. That is a textbook case for a **key-value / wide-column NoSQL store** (DynamoDB, Cassandra) which shards horizontally and gives fast primary-key lookups.

Column	Meaning
`short_key` (PK)	The 7-char base62 key — partition key.
`long_url`	The original destination.
`created_at / expires_at`	For cleanup and optional expiry.

Step 6 — The high-level design

KEY INSIGHT — CACHE IS THE WHOLE GAME ON READS

Traffic follows the 80/20 rule: a small set of links gets most clicks. Put a **Redis/Memcached** layer in front of the DB keyed by `short_key`. With a good hit rate, nearly every redirect is

served from memory in under a millisecond and the DB barely sweats at 116K reads/sec. Use LRU eviction so cold links age out.

Deep dive — 301 vs 302 redirect

This tiny choice has real consequences:

Redirect	Behavior	Consequence
301 Permanent	Browser caches it; future clicks skip your server.	Lowest load & latency, but you *lose analytics* on repeat clicks.
302 Temporary	Every click hits your server first.	Full click analytics, at the cost of more traffic.

If analytics matter, choose 302. If pure speed matters, choose 301. Naming that trade-off out loud is exactly the engineer’s move.

Trade-offs & lessons

WHAT TO TAKE AWAY

(1) The read/write ratio dictates the architecture — here it made caching the centerpiece. (2) Pre-computing work (KGS) beats doing it in the request path. (3) A simple access pattern is permission to pick a simple, horizontally-scalable NoSQL store rather than a relational DB you’ll fight to shard.

CASE 02

Design a Rate Limiter

The guardrail that keeps one caller from ruining the service for everyone.

The problem

Cap how many requests a client can make in a time window — e.g. “100 requests per minute per user.” Beyond that, reject with HTTP `429 Too Many Requests`. Rate limiting protects against abuse and DoS, controls infrastructure cost, and enforces fairness between tenants.

Step 1 — Requirements

Functional

- Limit requests per client (by user ID, IP, or API key).
- Reject excess requests clearly (429 + `Retry-After`).
- Rules configurable per endpoint / tier.

Non-functional

- Very low added latency — it sits in front of every request.
- Accurate under concurrency (no race-condition leaks).
- Works across a **fleet** of servers, not just one box.

Step 2 — Where does it live?

Client-side limits are trivially bypassed, so they don’t count. The limiter belongs **server-side**, and most naturally in the **API gateway** — a middleware every request already passes through before reaching business logic. That way one component enforces policy for all services.

Step 3 — The algorithms (know these cold)

Algorithm	How it works	Trade-off
Token bucket	A bucket holds up to N tokens, refilled at rate r. Each request removes one; empty = reject.	Allows short bursts; memory-cheap. *Used by Stripe, AWS.*
Leaky bucket	Requests enter a FIFO queue drained at a fixed rate.	Smooths output to a steady rate; but a full queue drops new requests and adds delay.
Fixed window	One counter per fixed window (e.g. each clock-minute).	Dead simple, but allows a *2× burst* straddling the window boundary.
Sliding window log	Store a timestamp per request; count those within the rolling window.	Perfectly accurate, but memory grows with request volume.
Sliding window counter	Weighted blend of current + previous window counts.	Great accuracy/memory balance. *Used by Cloudflare.*

KEY INSIGHT — TOKEN BUCKET IS THE DEFAULT ANSWER

Token bucket is the most common production choice because it's O(1) memory per client, allows natural bursts (users hate being throttled on a legitimate spike), and is trivial to reason about: two numbers per client — `tokens_left` and `last_refill_time`.

Step 4 — The fixed-window boundary problem

```
Limit = 5 / minute, fixed window:
  10:00:30  ****  (4 requests, end of minute one)
  10:01:00  ****  (4 requests, start of minute two)
  In the 60s spanning 10:00:30 -> 10:01:30 the user sent 8,
  yet each window counted <= 5. The limit was effectively doubled.
```

The sliding-window counter fixes this by weighting the previous window's count by how much of it still overlaps the rolling period — smoothing the boundary without storing every timestamp.

Step 5 — Making it work across the fleet

With dozens of gateway servers, a per-server in-memory counter fails: a user could hit the limit N times by landing on N different servers. The counters must be **shared**. Put them in a fast central store — **Redis** — using `INCR` plus a TTL (`EXPIRE`) for the window.

```
      request
        v
[Rate-limit middleware @ each gateway node]
        |  INCR user:123:window   (atomic)
        v
   [ Redis ]  <-- shared counters, single source of truth
        |
   under limit -> forward to service ; over -> 429
```

PITFALL — THE READ-MODIFY-WRITE RACE

If a node does `GET count → check → SET count+1` as three steps, two nodes can both read "99", both allow, and both write "100" — overshooting the limit. Fix with **atomicity**: Redis `INCR` is atomic, or wrap the check-and-increment in a single **Lua script** that Redis runs indivisibly. Never split the check from the update.

Deep dive — latency vs accuracy at scale

A central Redis is a network hop on every request. Two mitigations:

- **Local + sync:** each node keeps a local token bucket and periodically reconciles with Redis. Slightly less precise, but removes Redis from the hot path. Good when a small overshoot is acceptable.
- **Sticky routing:** route a given user consistently to the same node, so its local counter *is* authoritative. Costs you load-balancing flexibility.

And always tell the client the truth in headers: `X-RateLimit-Remaining`, `X-RateLimit-Reset`, and `Retry-After` on a 429 so well-behaved clients back off gracefully.

Trade-offs & lessons

WHAT TO TAKE AWAY

(1) “Distributed counter” problems are really **atomicity** problems — the moment state is shared, races appear. (2) Every accuracy gain (sliding log) costs memory; every latency gain (local counters) costs precision. Pick based on whether an occasional small overshoot actually hurts. (3) The centralized store is a dependency you must make highly available, or your limiter takes the whole API down with it.

CASE 03

Design a Distributed Unique ID Generator

Once your data is sharded, `AUTO_INCREMENT` dies. What replaces it?

The problem

A single database can hand out unique IDs with auto-increment. But split that database across 100 shards and there is no shared counter — two shards would both issue "1001." You need IDs that are **globally unique**, generated at high throughput, ideally **64-bit** (fits a `BIGINT`), and ideally **roughly time-sortable** so recent items sort together.

Step 1 — Requirements

Functional

- Generate globally unique IDs.
- IDs sortable by creation time (nice for feeds/ paging).
- Numeric & compact (64-bit).

Non-functional

- Highly available — if the generator stalls, writes stall.
- High throughput (tens of thousands/sec).
- Low coordination overhead.

Step 2 — Weigh the options

Approach	Pros	Cons
UUID v4	Generated locally, zero coordination, no collisions in practice.	128-bit (big), **not time-sortable**, random order fragments DB indexes.
Ticket server (one DB)	Simple, small numeric IDs.	Single point of failure & bottleneck. (HA needs two servers issuing odd/even.)
Snowflake (Twitter)	64-bit, time-sortable, no per-ID coordination, ~millions/sec.	Needs machine-ID assignment; sensitive to clock skew.

KEY INSIGHT — ENCODE THE ANSWER INTO THE BITS

Snowflake's trick is that a unique, sortable ID needs three things baked into one 64-bit number: **when** it was made (for sorting + uniqueness over time), **where** it was made (which machine, so machines never collide), and a **per-machine counter** (to disambiguate IDs made in the same millisecond). Combine those and no coordination between machines is needed at all.

Step 3 — The Snowflake bit layout

```
 1 bit        41 bits              10 bits        12 bits
+-----+--------------------+----------------+--------------+
```

```
|sign |   timestamp (ms)   |  machine id    |   sequence   |
| =0  |  since custom epoch |  (up to 1024)  | (0..4095/ms) |
+-----+--------------------+----------------+--------------+
  - 41 bits of ms  ~= 69 years of unique timestamps
  - 10 bits        = 1,024 distinct machines
  - 12 bits        = 4,096 IDs per machine per millisecond
  => up to ~4.09 million IDs/sec per machine
```

Because the timestamp occupies the high-order bits, sorting IDs numerically sorts them (approximately) by creation time — exactly what feeds and pagination want.

Deep dive — the two failure modes

1. Clock moving backwards

Servers sync time via NTP, which can occasionally step the clock *backward*. If that happens, a new ID could reuse a timestamp already used — risking a duplicate. Handling options, in order of strictness:

- **Wait it out:** if the clock is behind the last-seen timestamp, block until it catches up. Safe, but pauses issuance briefly.
- **Reject:** throw an error and let the caller retry. Simple, pushes the problem up.
- **Logical clock:** track the last timestamp used and never go below it, advancing a logical counter instead. Most robust.

PITFALL — RUNNING OUT OF SEQUENCE BITS

If a machine issues more than 4,096 IDs within a single millisecond, the 12-bit sequence overflows. The correct behavior is to **spin-wait until the next millisecond** and reset the sequence to 0 — never wrap around and collide within the same ms.

2. Assigning machine IDs

Every generator needs a unique 10-bit ID, and no two may share one. Hard-coding is fragile. The standard fix is a coordination service like **ZooKeeper**: on startup each node claims an unused machine ID from a shared registry and releases it on shutdown. This is the *only* coordination Snowflake needs — and it happens once per boot, not once per ID.

Trade-offs & lessons

WHAT TO TAKE AWAY

(1) UUID vs Snowflake is a real fork: pick UUID when you need *zero* infrastructure and don't care about ordering; pick Snowflake when sortability and compactness matter. (2) "Time-sortable" is worth engineering for — it makes downstream feeds and cursor pagination almost free. (3) Anything that leans on wall-clock time must have an explicit plan for the clock going backwards; that edge case is where naive designs silently produce duplicates.

CASE 04

Design a News Feed / Timeline

Twitter, Instagram — where the "celebrity problem" forces a hybrid.

The problem

When you open the app, you see a feed of recent posts from everyone you follow, newest-ish first. Two operations: **publish** a post, and **read** your home timeline. The tension between making those two cheap is the entire design.

Step 1 — Requirements

Functional

- Publish a post to your followers.
- Load a home timeline (merged posts of everyone you follow).
- Support pagination / infinite scroll.

Non-functional

- Extremely **read-heavy**; feed load must feel instant.
- Eventual consistency is fine (a post appearing a second late is OK).
- Highly available; huge fan-out per popular user.

Step 2 — The core decision: fan-out on write vs read

Model	On publish	On read	Weakness
Fan-out on write (push)	Insert the post ID into every follower's precomputed feed.	Just read your ready-made feed — O(1), instant.	A user with 50M followers triggers 50M writes per post ("write amplification").
Fan-out on read (pull)	Just store the post once.	Query all the people you follow, merge & sort at read time.	If you follow 5,000 accounts, every feed load is an expensive merge.

KEY INSIGHT — THE HYBRID IS THE REAL ANSWER

Neither pure model survives at scale. Production systems **push for normal users** (fan-out on write, so 99% of feeds are precomputed and instant) but **pull for celebrities** (don't fan-out a superstar's post to 50M feeds). At read time, you merge your precomputed feed with the latest posts from the handful of celebrities you follow. Best of both: cheap writes for the few accounts that would blow up write volume, cheap reads for everyone else.

Step 3 — High-level design

```
publish post
    |
[Post Service] --> [ Post DB ]  (source of truth for content)
    |
    +--> [Fan-out Service]
            |  normal author?  -> push post_id into each
```

```
            |                        follower's feed cache
            |  celebrity?      -> skip fan-out (handled at read)
            v
       [ Feed Cache ]  (Redis list per user: [post_id, post_id, ...])

read feed
    |
[Timeline Service]
    |  1) read your precomputed feed_cache (post ids)
    |  2) merge in recent posts from celebrities you follow
    |  3) hydrate: fetch post content from cache/Post DB
    v  return page
```

Deep dive — hydration and the graph

The feed cache stores only **post IDs**, not full content — that keeps it small and avoids duplicating a viral post a million times. Turning IDs into displayable posts (author, text, media, like counts) is **hydration**, done at read time from a heavily-cached Post service. This separation means editing or deleting a post updates one record, not a million copies sitting in feeds.

To fan out, you need each author's follower list — served by a **graph/follow service**, itself cached because it's read constantly. Pagination uses a **cursor** (the last post ID / timestamp seen) rather than offset, so scrolling stays O(1) even deep into the feed.

PITFALL — TREATING THE CELEBRITY CASE AS AN AFTERTHOUGHT

A design that only does fan-out on write looks elegant in a demo and then melts the moment a user with millions of followers posts. Interviewers (and reality) probe exactly this. Always state the follower-count threshold that flips an account from push to pull.

Trade-offs & lessons

WHAT TO TAKE AWAY

(1) When two operations pull in opposite directions, the answer is often a **hybrid keyed on a property** (here, follower count) rather than one pure strategy. (2) Store references (IDs), not copies, and hydrate late — it keeps caches small and edits cheap. (3) "Eventual consistency is acceptable" is a powerful license: it's what lets feeds be precomputed asynchronously at all.

CASE 05

Design a Chat System

WhatsApp, Messenger — where HTTP request/response stops being enough.

The problem

Deliver messages between users in real time: 1-to-1 and group chats, online presence, delivery/read receipts, and offline delivery via push notification. The twist that makes this different from everything before: the **server must push to the client unprompted** the instant a message arrives — you can't wait for the receiver to poll.

Step 1 — Requirements

Functional

- 1:1 and group messaging.
- Online / last-seen presence.
- Delivery + read receipts; offline delivery.

Non-functional

- Low latency; real-time push.
- Consistent **message ordering** within a conversation.
- Reliable — no lost messages; huge write volume.

Step 2 — The connection: why WebSockets

Plain HTTP is client-initiated, so the server can't start a message. Options to get server-push are long-polling (works, wasteful) and **WebSocket** — a single, persistent, full-duplex TCP connection over which either side can send at any time. WebSocket is the standard answer: the client opens one to a chat server on login and keeps it open.

Step 3 — Stateless vs stateful services

KEY INSIGHT — SPLIT THE STATELESS FROM THE STATEFUL

Auth, profile, and group-management are ordinary **stateless** HTTP services — easy to scale behind a load balancer. But the **chat servers holding live WebSocket connections are stateful**: a given user's socket lives on one specific server. To deliver a message you must know *which* chat server the recipient is connected to. That mapping (user → server) is the heart of the system, usually kept in a fast store like Redis and updated as users connect/disconnect.

Step 4 — Message flow (1:1)

```
|     B online on server 7 --> route msg to server 7 --> push to B
|     B offline           --> store only + trigger [Push Notif]
```

Step 5 — Storing messages at scale

Message volume is enormous and the access pattern is simple: “fetch the recent messages of conversation X in order.” That points to a **wide-column NoSQL store** (Cassandra / HBase) partitioned by conversation ID, with a clustering key that sorts by time. Message IDs should be **time-sortable** (recall Case 03 — a Snowflake-style ID) so ordering within a conversation is well-defined even when clocks differ across servers.

Deep dive — scaling millions of live connections

Each chat server can hold only so many open sockets (tens of thousands to ~1M with tuning). Serving hundreds of millions of users therefore needs **many** chat servers plus the routing layer above. Presence adds its own load: clients send periodic **heartbeats**; a missed heartbeat flips a user to offline; and status changes are broadcast to that user’s contacts via **pub/sub**. Broadcasting presence naively to everyone is a scaling trap — only push updates to users who actually have the person on screen.

PITFALL — ASSUMING A MESSAGE SENT IS A MESSAGE DELIVERED

Networks drop packets and phones lose signal. Guarantee delivery with **acknowledgements**: the recipient ACKs receipt; if the sender’s server gets no ACK, it retries. Retries can cause duplicates, so give every message a unique ID and have clients **de-duplicate** on it. This ACK + retry + dedup triad is how you approximate exactly-once over an unreliable network.

Group chat

For small groups, treat a group message as fan-out to each member’s queue/connection (like Case 04’s push model). Very large groups lean toward a pull/read model to avoid write amplification — the same fork you saw in the news feed.

Trade-offs & lessons

WHAT TO TAKE AWAY

(1) The moment you need server-initiated push, reach for **persistent connections** — and accept that they make part of your system stateful, which is the hard part to scale. (2) A user→server routing table is unavoidable and must be fast and highly available. (3) “Reliable delivery” over an unreliable network always decomposes into ACK + retry + dedup with idempotent message IDs.

CASE 06

Design a Notification System

Push, SMS, and email at scale — a masterclass in queues and retries.

The problem

Let any internal service trigger a notification to a user across several channels — mobile push (APNs for iOS, FCM for Android), SMS (via Twilio), and email (via a provider like SES). Handle user opt-outs, avoid spamming, and above all **don't lose notifications** even when a third-party provider is briefly down.

Step 1 — Requirements

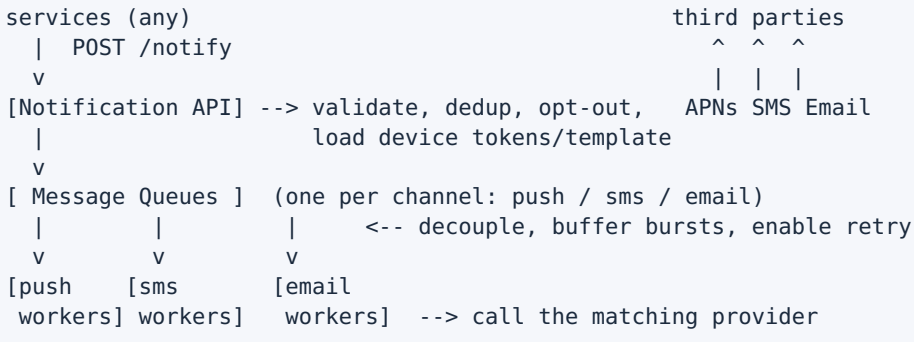
Functional

- Send across push / SMS / email channels.
- Respect per-user opt-out & preferences.
- Templates; triggered by many services.

Non-functional

- **Reliable** — at-least-once delivery, no silent drops.
- Scalable; absorb sudden bursts.
- Resilient to third-party provider outages.

Step 2 — High-level design



KEY INSIGHT — THE QUEUE IS THE RELIABILITY

Putting a durable **message queue between the API and the workers** is what makes this system robust. It (1) absorbs traffic spikes so a flood of events never overwhelms providers, (2) **decouples** your API's success from a provider's availability — the API just enqueues and returns, and (3) enables clean **retries**: if a provider call fails, the message goes back on the queue or into a retry queue instead of vanishing.

Step 3 — A channel per queue

Each channel has different throughput, latency, and failure behavior, so give each its own queue and worker pool. If the SMS provider slows down, push and email keep flowing untouched. Workers pull, render the template, call the provider, and record the outcome.

Deep dive — reliability: retries, dead letters, and dedup

“Don’t lose notifications” is the whole game. The machinery:

- **Persist a notification log** the moment a request arrives, so state survives crashes and you can audit what was sent.
- **Retry with backoff** on transient provider failures; after N attempts move the message to a **dead-letter queue** for inspection rather than retrying forever.
- **At-least-once + dedup:** guaranteeing delivery means you may send twice. Attach a **dedup/idempotency key** so a repeated attempt for the same logical notification is suppressed — approximating exactly-once.

PITFALL — SPAMMING THE USER INTO MUTING YOU

Reliability pushes toward sending *more* (retries, at-least-once). Left unchecked, a bug or a retry storm buries the user in duplicates and they disable notifications entirely — a permanent loss. Add a **rate limit per user** and collapse/notification-grouping so a burst of events becomes one digest, not fifty pings.

NOTE — ORDERING IS NOT GUARANTEED

With parallel workers and independent retries, notifications can arrive out of order. For most notifications that’s fine. If order matters for a specific stream, you need per-user sequencing on a single partition — a real cost, so only pay it where it’s truly required.

Trade-offs & lessons

WHAT TO TAKE AWAY

(1) A queue between request and delivery is the single most important pattern for reliable async work — it turns provider outages from data loss into mere delay. (2) “At-least-once” is usually the honest guarantee; you buy back “exactly-once” behavior with idempotency keys, not with wishful thinking. (3) Reliability and user-friendliness pull in opposite directions here — the rate limiter is what reconciles them.

CASE 07

Design a Web Crawler

The bot that feeds a search engine — a lesson in politeness and dedup.

The problem

Start from a set of seed URLs, download each page, extract the links inside, and repeat — discovering and storing a large chunk of the web. Simple as a loop; brutal at billions of pages, where you must be polite to servers, avoid crawling the same thing twice, and not fall into infinite traps.

Step 1 — Requirements

Functional

- Crawl from seeds, extract & follow links.
- Store page content for indexing.
- Respect `robots.txt`.

Non-functional

- **Scalable** to billions of pages.
- **Polite** — never hammer one host.
- Robust to traps/malformed pages; refreshable.

Step 2 — Estimate

Crawl ~1B pages/month at ~500 KB each → **~500 TB/month** of raw content. Storage and bandwidth, not CPU, dominate — and de-duplication directly saves both.

Step 3 — The components

KEY INSIGHT — THE URL FRONTIER IS THE WHOLE DESIGN

The frontier isn't just a queue; it enforces two competing goals at once. **Priority** (crawl important/fresh pages first) is handled by front queues that rank URLs. **Politeness** (don't overload any single host) is handled by back queues where *each host maps to exactly one queue*, and a worker pulling from that queue waits a delay between requests. Getting these

two to coexist — important pages soon, but never rudely fast to one server — is the crux of crawler design.

Deep dive — two kinds of “have I seen this?”

- **URL seen?** Before adding a URL to the frontier, check whether it’s already known. With billions of URLs, an exact set is huge, so use a **Bloom filter**: it answers “definitely new” or “probably seen” in tiny memory. A false positive occasionally skips a real URL — an acceptable price for the space saved.
- **Content seen?** Different URLs often serve identical content (mirrors, session IDs). Hash each page’s content and skip storing/indexing duplicates — saving that 500 TB from ballooning.

PITFALL — SPIDER TRAPS

Some sites generate infinite URL spaces (endless calendars, faceted filters like `?page=1&sort=a&sort=b...`). A naive crawler follows them forever and never finishes. Defend with limits: max crawl depth, max URL length, per-domain page caps, and heuristics that detect near-identical generated pages.

Politeness & robots.txt

Fetch and cache each site’s `robots.txt`, honor its disallow rules and `crawl-delay`, and identify your bot honestly via User-Agent. Politeness is both etiquette and self-interest — hammered servers block you.

Trade-offs & lessons

WHAT TO TAKE AWAY

(1) When a design has two goals that fight (priority vs politeness), the answer is a **structured queue that separates the two concerns** into distinct layers. (2) **Bloom filters** are the go-to whenever an exact membership set would be too big and a rare false positive is survivable. (3) The web is adversarial and messy; robustness (traps, malformed HTML, dedup) is not polish — it’s what lets the crawl terminate at all.

CASE 08

Design a Distributed Key-Value Store

Amazon Dynamo — the deepest case: CAP, quorum, and conflict resolution.

The problem

Build a store exposing just `put(key, value)` and `get(key)`, but spread across hundreds of commodity machines so it holds far more data than one server and survives machines dying. This is the case where the distributed-systems theory from your basics becomes concrete.

Step 1 — Requirements & the CAP choice

Functional

- `get(key)` / `put(key, value)`.
- Data spread over many nodes.
- Tunable consistency.

Non-functional

- Highly scalable & available.
- Partition-tolerant (nodes/networks fail).
- Predictable low latency.

THE DEFINING DECISION (CAP)

Networks partition; you must pre-decide C or A. Dynamo-style stores choose **AP**: always accept reads and writes (stay available) and reconcile any resulting inconsistency later. That single choice — "always on, eventually consistent" — shapes every mechanism below.

Step 2 — Partitioning: consistent hashing

Spreading keys with plain `hash(key) % N` is a trap: change N (add/remove a node) and nearly every key remaps, forcing a massive reshuffle. **Consistent hashing** places both nodes and keys on a ring; a key belongs to the next node clockwise. Add or remove a node and only the keys in that arc move — about 1/N of the data.

KEY INSIGHT — VIRTUAL NODES FIX LUMPINESS

With few nodes the ring is uneven — some nodes own huge arcs. Give each physical node many **virtual nodes** scattered around the ring. Load evens out, and when a node dies its share is spread across *many* others instead of dumping onto one unlucky neighbor.

Step 3 — Replication & tunable consistency (N, W, R)

Each key is replicated to the next **N** nodes on the ring. Consistency is tuned with two knobs: a write must be acknowledged by **W** replicas; a read must gather **R** replicas.

```
If  W + R > N   -> read set and write set overlap
                  -> a read always sees the latest write
                     (strong-ish consistency)
Examples (N=3):
   W=3, R=1  -> fast reads, slow/less-available writes
```

```
W=1, R=3  -> fast writes, slower reads
W=2, R=2  -> balanced, and 2+2 > 3 so still consistent
```

Deep dive — staying available when nodes fail

- **Sloppy quorum + hinted handoff:** if a target replica is down, write to the next healthy node instead, tagged with a “hint” that it belongs to the down node. When that node returns, the data is handed back. Writes never block on a dead replica.
- **Anti-entropy with Merkle trees:** replicas drift out of sync after failures. Rather than compare every key, each replica builds a **Merkle tree** (a hash tree) of its data; two replicas compare just the top hashes and drill down only where they differ — syncing with minimal data transfer.
- **Gossip protocol:** nodes periodically exchange membership/health info peer-to-peer, so the cluster learns who’s alive without a central coordinator.

PITFALL — CONCURRENT WRITES AND CONFLICT RESOLUTION

Because it’s AP, two clients can write the same key on different sides of a partition. Which wins? **Vector clocks** tag each version with its causal history so the system can tell “B descends from A” (keep B) from “A and B are concurrent” (a true conflict). Concurrent conflicts are surfaced to the client to reconcile, or resolved by a simple **last-write-wins** policy — simpler, but it can silently drop a write. Choosing between them is a real design call.

Trade-offs & lessons

WHAT TO TAKE AWAY

(1) CAP isn’t academic — picking AP is what forces quorums, hinted handoff, and conflict resolution to exist. (2) **N, W, R** turn consistency into a dial you set per workload, not a fixed property. (3) Nearly every mechanism here (virtual nodes, Merkle trees, gossip) exists to make failure *ordinary and cheap* rather than catastrophic — the real definition of a distributed system.

CASE 09

Design a Video Streaming Service

YouTube, Netflix — where the CDN and a transcoding pipeline run the show.

The problem

Let users upload videos and let millions stream them smoothly on any device and any bandwidth. Two very different pipelines meet here: a heavy **write path** (ingest + process one video) and an enormous **read path** (serve that video to the world). Storage is large, but **egress bandwidth is the true cost driver**.

Step 1 — Requirements

Functional

- Upload & process videos.
- Stream smoothly across devices/networks.
- Search, metadata, thumbnails.

Non-functional

- Low start-up latency, no buffering.
- Highly available & scalable.
- Cost-efficient at massive egress.

Step 2 — The upload / processing pipeline

```
creator
  |  chunked, resumable upload
  v
[ Blob Storage ]  (raw original, e.g. S3)
  |
  v
[ Transcoding Pipeline ]  (orchestrated via a DAG + queue)
  |  - split video into chunks
  |  - transcode each to many resolutions/bitrates/codecs
  |     (240p, 480p, 720p, 1080p, 4K ...)
  |  - package into HLS/DASH segments + manifest
  v
[ Processed Blob Storage ] --> push popular content to [ CDN ]
  |
[ Metadata DB updated: video is ready ]
```

KEY INSIGHT — TRANSCODING IS A PARALLEL DAG

You never store just one copy of a video. You produce **many** — every resolution × bitrate × codec — so any device on any network gets a suitable stream. Because a video splits into independent chunks, transcoding is **massively parallelizable**: model it as a DAG of stages (split → transcode → package) driven by a message queue, running thousands of chunk-jobs at once. This is why a long video is ready in minutes, not hours.

Step 3 — The streaming path & the CDN

On playback the client fetches a **manifest** listing the available quality levels and their segment URLs, then streams short segments — served from the nearest **CDN edge**, not your origin.

KEY INSIGHT — THE CDN IS THE READ ARCHITECTURE

Serving 500 TB of video from a central origin would be ruinously slow and expensive. The **CDN** caches content on edge servers physically near users, cutting latency and offloading nearly all traffic from the origin. Crucially, viewing is 80/20: a small fraction of videos gets most views. So you push only **popular** content to the CDN and serve the long tail from origin — capturing most of the benefit for a fraction of the cost.

Deep dive — adaptive bitrate streaming (ABR)

Networks fluctuate. With **HLS** or **DASH**, the client measures its throughput and, *segment by segment*, picks the highest quality that won't stall — dropping to 480p when the connection dips, climbing back to 1080p when it recovers. Playback stays smooth instead of freezing. This is why you transcoded all those renditions up front: ABR needs a ladder of qualities to choose from.

PITFALL — TREATING UPLOAD AS A SINGLE REQUEST

A 4 GB upload over flaky mobile will fail and force a restart-from-zero — infuriating. Use **chunked, resumable uploads**: split the file client-side and upload parts independently, retrying only the failed chunk. Same lesson as the crawler and chat cases — big or unreliable operations must be broken into retryable pieces.

Trade-offs & lessons

WHAT TO TAKE AWAY

(1) When read and write paths have wildly different shapes, design them as **two separate systems** that meet at storage. (2) A CDN plus the 80/20 rule is the canonical way to serve huge static content affordably — cache the hot, origin the cold. (3) Pre-computing variants (all those renditions) is what makes the smart runtime behavior (ABR) cheap — a recurring theme: move work off the hot path.

CASE 10

Design a Ride-Sharing / Nearby Service

Uber, Lyft — the finale: geospatial indexing under a firehose of writes.

The problem

Show a rider the drivers near them, match a ride request to a good driver, and track everyone's location live on the map. The defining challenges: answering "who is near this point?" fast, while ingesting a **relentless stream of location updates** from every moving driver.

Step 1 — Requirements

Functional

- Drivers stream live location.
- Riders see nearby drivers; request a ride.
- Match rider ↔ driver; show ETA.

Non-functional

- Real-time; low-latency "nearby" search.
- Handle a huge **write** volume of updates.
- Highly available; consistent matching.

Step 2 — The write firehose

If millions of drivers each send a location every ~4 seconds, that's **millions of writes per second**. Persisting each to a disk-based DB is hopeless. Keep current driver locations in an **in-memory store** (e.g. Redis) that's constantly overwritten; the DB is for trip records, not the live-position stream.

Step 3 — The core challenge: geospatial indexing

"Find drivers within 2 km" over raw lat/long would scan everyone — impossible at scale. You need a spatial index that groups nearby points so you only examine a small region.

Approach	Idea	Trade-off
Geohash	Encode lat/long into a short string; shared prefix ≈ spatial closeness. Query a cell and its 8 neighbors.	Simple, DB-friendly, widely used. Edge cases at cell boundaries.
Quadtree	Recursively split space into 4; dense areas subdivide deeper. Leaves hold the points.	Adapts to density (dense city vs empty highway). In-memory; rebuilt periodically.
H3 / S2	Hierarchical hex/cell grids used in real production systems (Uber's H3).	Robust and battle-tested; more complex to implement.

KEY INSIGHT — TURN "NEARBY" INTO A CELL LOOKUP

The whole trick is converting a costly distance search into a cheap **cell membership** lookup. Bucket every driver into a spatial cell (geohash/quadtree). To find who's near a rider, compute the rider's cell, gather drivers in that cell plus adjacent cells, and only then compute exact distances on that small candidate set. Search cost drops from "everyone" to "this neighborhood."

Step 4 — High-level design

```
driver app --location every 4s--> [ Location Service ]
                                        |  update in-memory
                                        v  geo-index (Redis)
rider app --"find nearby"--> [ Matching Service ]
                                  |  1) rider's cell + neighbors
                                  |  2) candidate drivers
                                  |  3) rank by ETA/road distance
                                  |  4) dispatch offer to best driver
                                  v
                            [ Trip DB ] (durable trip records)
```

Deep dive — the matching race condition

PITFALL — ASSIGNING ONE DRIVER TO TWO RIDERS

Two riders' searches can surface the same idle driver at the same instant. Without care, both get matched to one car. Guard the assignment with a **lock or an atomic state transition** on the driver: the first request flips the driver to `ASSIGNED` and any concurrent attempt fails and moves to the next candidate. The lookup can be eventually consistent, but the *assignment* must be strongly consistent — know which parts of your system can tolerate staleness and which cannot.

Why quadtrees shine for density

A fixed geohash cell holds thousands of drivers downtown but zero on a rural road — unbalanced. A **quadtree** subdivides only where it's crowded, so each leaf holds a manageable count regardless of area. The cost is maintenance: as drivers move, the tree is rebuilt periodically rather than updated perfectly in real time — an acceptable trade since "nearby" tolerates being a few seconds stale.

Trade-offs & lessons

WHAT TO TAKE AWAY

(1) A high-frequency, disposable data stream (live positions) belongs **in memory**, kept separate from durable records (trips). (2) Expensive searches become cheap when you pre-bucket data by the dimension you search on — here, space. (3) Be surgical about consistency: reads/lookups can be eventually consistent, but the money-moving step (the match) must be atomic. Naming which is which is the mark of a mature design.

APPENDIX

The universal framework

A checklist to attack any system design problem you've never seen.

You now have ten worked examples. The point was never the ten answers — it was the **repeatable method** underneath them. Run this checklist on any new problem and you will never freeze at a blank page.

1. Scope & requirements (don't skip)

- Ask what it must **do** (functional) before how well (non-functional).
- Pin the non-functionals that decide architecture: read/write ratio, scale, latency budget, availability target, and the consistency it truly needs.
- State assumptions out loud. Narrowing scope is a strength, not a dodge.

2. Estimate before you draw

- Compute QPS (reads and writes separately), storage over time, and bandwidth.
- The numbers pick the tools: a 100:1 read ratio screams "cache"; 91 TB screams "shard."

3. Define the API & data model

- Write the handful of core operations. They reveal the access pattern.
- A simple access pattern → NoSQL KV/wide-column. Rich relations/transactions → SQL.

4. High-level design, then deep dive

- Draw a working end-to-end path first — boxes and arrows — before perfecting any one box.
- Then attack the 1–2 genuinely hard sub-problems. That is where the interview and the real system are won.

5. Name your trade-offs

Every choice costs something. Saying "I choose AP here, accepting stale reads to stay available" is worth more than any diagram. There is no perfect design — only one whose costs you chose on purpose.

The patterns that recurred across all ten

Pattern	Where you saw it
Cache the read-heavy path	URL shortener, feed, video CDN
Move work off the hot path (pre-compute)	KGS keys, precomputed feeds, transcoded renditions

Queue to decouple + retry	Notifications, crawler frontier, transcoding
Hybrid keyed on a property	Feed & group chat: push for the many, pull for the few
ACK + retry + dedup = reliable-once	Chat delivery, notifications
Consistent hashing for elastic sharding	KV store; implicit in many caches
Bucket data by what you search on	Geospatial cells in ride-sharing
Atomicity kills races	Rate limiter counter, ride matching
Pick consistency per operation	Ride-sharing: loose lookup, strict assignment

THE ONE HABIT TO KEEP

For every component you add, ask three questions: **What happens when it's slow? When it's down? When traffic 10×'s?** A design that answers those for each box is a senior design. Everything in this book was really just those three questions, asked over and over.

Now close the book and design something. Pick a system you use daily — a food-delivery app, a cloud file store, an online multiplayer game — and run the five steps cold. That gap between your attempt and a worked solution is exactly where you turn into a system engineer.